Department of Computer Science and Engineering Assignment Questions

Course Code & Name : CSA0927 – Programming in Java

Assignment Title:	Government Public Grievance Management System using Java
Course Outcome(s) – CO:	**CO1:** Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4) **CO2:** Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications.(BL3,BL4) **CO3:** Build multithreaded Java programs by applying thread priorities, synchronisation, and inter-thread communication mechanisms to achieve concurrent program execution and use exception handling techniques (BL3,BL4)
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse
SDG Mapping (SDG 1-17)	SDG 16 – Peace, Justice and Strong Institutions SDG 9 – Industry, Innovation and Infrastructure SDG 10 – Reduced Inequalities SDG 11 – Sustainable Cities and Communities

Problem Statement : A district government office receives numerous public complaints related to roads, water supply, sanitation, streetlights, public transport, and civic services. The existing manual process makes it difficult to register complaints, assign them to the appropriate departments, track their status, and monitor pending cases. The government department plans to develop a Java-based Public Grievance Management System to automate the complaint-handling

process. The system should maintain citizen details, complaint records, departments, responsible officers, priority levels, and complaint status.Different users such as citizens, officers, supervisors, and administrators should have different roles and responsibilities. The application should apply classes, encapsulation, inheritance, method overriding, and polymorphism to represent these entities.Citizen records, complaints, department details, and complaint history should be managed using the Java Collection Framework, including ArrayList, HashSet, and HashMap, along with generics and iterators.Since multiple citizens may submit complaints simultaneously, the system should implement multithreading, synchronization, and inter-thread communication to process requests safely and avoid conflicts. Appropriate exception handling should be provided for duplicate complaints, invalid inputs, unavailable departments, and incorrect status updates.The final application should provide a menu-driven interface for citizen registration, complaint submission, officer assignment, status updates, complaint tracking, and generation of department-wise and pending-complaint reports.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
1. Requirement Analysis, Java Fundamentals & Program Logic	**15**	Clearly identifies citizens, grievances, departments, officers, priorities, and workflows; uses appropriate data types, operators, conditional statements, loops, and decision-making structures correctly and efficiently.	Most requirements and workflows are identified; Java fundamentals and control structures are used correctly with minor errors.	Basic requirements are identified; fundamental constructs are used with some logical errors or inefficiencies.	Requirements are poorly understood; frequent errors in Java fundamentals and program logic affect correctness.

2. Class Design, Encapsulation, Inheritance & Polymorphism	**20**	Well-structured classes for citizens, officers, departments, grievances, and management; strong encapsulation and data hiding; appropriate inheritance, method overriding, and runtime polymorphism are implemented effectively.	Classes are reasonably designed with appropriate encapsulation and inheritance; polymorphism is implemented with minor limitations.	Basic classes and inheritance are present, but encapsulation or polymorphic behavior is incomplete.	Poor class organization; weak encapsulation; inheritance or polymorphism is absent or incorrectly implemented.
3. Java Collection Framework, Generics & Data Management	**20**	Effectively uses List, ArrayList, Set, HashSet, Map, and HashMap with generics and iterators to store, search, update, and process citizen, grievance, department, and complaint-history records.	Appropriate collections and generics are used with minor implementation or retrieval issues.	Collections are used for basic storage and retrieval, but some data structures or iterator operations are incomplete.	Collections, generics, or iterators are largely missing, incorrectly implemented, or inefficiently used.
4. Multithreading, Synchronization, Exception Handling & Menu-Driven Functionality	**25**	Multiple threads effectively process concurrent grievance submissions, officer assignments, and status updates; synchronization and inter-thread communication prevent conflicts; exception handling manages duplicate complaints, invalid inputs, unavailable departments, and incorrect status	Multithreading, synchronizatio n, exception handling, and menu operations are mostly implemented correctly with minor issues.	Basic multithreading and exception handling are present, but synchronization, communication mechanisms, or some menu functions are incomplete.	Multithreading or synchronization is absent/non-function al; exception handling is inadequate and major menu operations are missing or unreliable.

		updates; all menu operations function reliably.			
5. System Integration, Reports, Documentation & Overall Quality	**10**	All modules are fully integrated; complaint tracking, departmental allocation, pending-grievance analysis, and performance reports are generated correctly; code is organized, readable, documented, and professionally presented.	Modules are well integrated and reports are mostly correct; code organization and documentation are satisfactory.	Basic integration and reporting are achieved, but some reports, documentation, or code quality need improvement.	Poor integration; reports are missing or incorrect; code is difficult to understand and documentation is inadequate.
6. Reflection: Design Decisions, SDG Relevance & Learning Outcomes	**10**	Provides thoughtful justification of design decisions; clearly connects the system with **SDG 16 – Peace, Justice and Strong Institutions**, along with relevant SDGs such as SDG 9, 10, and 11; provides specific reflection on challenges, problem-solving, and learning outcomes.	Provides reasonable justification of design choices and SDG relevance; discusses challenges and learning with adequate detail.	Reflection is present but generic; limited justification of design decisions, SDG relevance, or learning outcomes.	Reflection is missing or lacks meaningful discussion of design decisions, SDG relevance, challenges, or learning outcomes.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA0927 – PROGRAMMING IN JAVA

Assignment: Government Public Grievance Management System

1. Problem Statement

In a district office, citizens regularly report faults in essential services such as damaged roads, irregular water supply, poor sanitation, non-working streetlights and transport difficulties. When these reports are handled through notebooks or separate files, officers may not know which cases are pending, who is responsible, or whether a citizen has received a response. This project proposes a Java application that records each grievance once, sends it to the suitable department and makes its progress visible until completion.

The application keeps related information about residents, service departments, officers, priorities, current states and previous actions in one controlled system. It also provides different capabilities for citizens, officers, supervisors and administrators. The implementation was selected to demonstrate the object-oriented, collection

Scope

The prototype is a console-based application. It focuses on the core complaint lifecycle: citizen registration, complaint submission, validation, department allocation, officer assignment, status updates, tracking and report generation. The design can later be connected to a database and a web interface without changing the main domain model.

2. Objective

This project aims to create a dependable Java prototype for receiving, organizing and following up public-service complaints. It converts the requirements into a set of cooperating classes, uses typed collections for record management, protects shared data when submissions occur together, and reports errors in a controlled manner. The program also produces useful summaries for departmental monitoring.

Objective	**Evidence in implementation**
CO1: Object-oriented design	User is abstract; Citizen, Officer, Supervisor

	and Administrator inherit from User. Private fields provide data hiding. role() is overridden.
CO2: Collections and generics	HashMap stores citizens, departments and complaints; HashSet detects duplicates; ArrayList stores history; stream and collection traversal support reports.
CO3: Concurrency and safety	Two submission threads run concurrently. synchronized service methods protect shared state, while wait()/notifyAll() demonstrate inter-thread communication.
Application functionality	Registration, complaint submission, assignment, status update, tracking and reports are implemented and tested.

3. Requirements and Environment Used

3.1 Functional Requirements

• Register citizens with a unique citizen ID and valid contact details.

• Store departments and verify that the selected department is available.

• Submit complaints with category, description, priority and department.

• Reject duplicate complaints using a complaint fingerprint.

• Assign an officer only when the officer belongs to the complaint department.

• Update status only in the forward direction: REGISTERED → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED.

• Track an individual complaint by its generated ID.

• Generate pending-complaint and department-wise reports.

• Process simultaneous complaint submissions without corrupting shared data.

3.2 Non-Functional Requirements

The application should be readable, maintainable, modular and reasonably extensible. Shared data must be protected from race conditions. Invalid operations must produce clear messages rather than terminating the application unexpectedly. The interface should be simple enough for a console demonstration and the design should support later migration to a database-backed web application.

3.3 Environment Used

Item	Specification
Programming language	Java 21
Compiler/runtime	OpenJDK 21; javac and java
Development style	Object-oriented, console-based prototype
Data structures	HashMap, HashSet, ArrayList, generics, streams
Concurrency	Thread, synchronized methods, wait(), notifyAll(), join()
Documentation	Microsoft Word-compatible DOCX; Times New Roman; justified paragraphs
Operating system used for testing	Ubuntu Linux 24.04 sandbox

4. Design / Proposed Solution

The design separates the data objects from the operations that coordinate them. User is the common parent type for the people who interact with the application. Citizen, Officer, Supervisor and Administrator add role-specific meaning. Department describes an office that handles a service area, while Complaint represents the main transaction. GrievanceService acts as the central coordinator for validation, storage, assignment, status changes and reports.

4.1 Main Classes

Class	Responsibility	OOP feature
User	Common identity and role behaviour	Abstract class, encapsulation
Citizen	Represents a complaint submitter	Inheritance, overriding
Officer	Represents a department officer	Inheritance, department association
Supervisor	Represents supervisory access	Inheritance, overriding
Administrator	Represents administrative access	Inheritance, overriding
Department	Stores department identity and availability	Encapsulation
Complaint	Stores grievance details and status	Encapsulation, state validation

GrievanceService	Coordinates registration, submission, assignment, tracking and reporting	Synchronization, collection management

4.2 Data-Structure Design

Collection	Key / element	Purpose
HashMap<String, Citizen>	Citizen ID → Citizen	Fast citizen lookup and uniqueness
HashMap<String, Department>	Department ID → Department	Department availability and lookup
HashMap<Integer, Complaint>	Complaint ID → Complaint	Complaint tracking and report generation
HashSet<String>	Complaint fingerprint	Duplicate complaint detection
ArrayList<String>	History event strings	Sequential audit history

4.3 Workflow

A citizen is registered after input validation. During submission, the service checks registration, required fields, department availability and duplicate fingerprint. A valid complaint receives an atomic generated ID and is stored. An officer is assigned only from the relevant department. Status changes are checked against the permitted forward workflow. Reports read the protected complaint collection and group or filter records as required.

5. Algorithm, Pseudocode and Flowchart

5.1 Complaint Submission Pseudocode

START

Initialize Citizen ArrayList

Initialize Complaint ArrayList

Initialize Complaint ID HashSet

Initialize Department HashMap

Create available departments:

Road

Water

Sanitation

Streetlight

Transport

DISPLAY Main Menu

REPEAT until user selects Exit

DISPLAY:

1. Register Citizen
2. Submit Complaint
3. Assign Officer
4. Update Complaint Status
5. Track Complaint
6. Display All Complaints
7. Pending Complaint Report
8. Department-wise Report
9. Display Departments
10. Exit

READ user choice

IF choice = 1 THEN

READ Citizen ID, Name and Phone

CREATE Citizen object

ADD Citizen to ArrayList

DISPLAY "Citizen registered successfully"

ELSE IF choice = 2 THEN

READ Complaint ID and Citizen ID

FIND Citizen

```
    IF Citizen does not exist THEN
        DISPLAY "Citizen not found"
    ELSE
        READ Category, Description and Priority

        IF Priority is invalid THEN
            DISPLAY "Invalid priority"
        ELSE IF Complaint ID already exists THEN
            DISPLAY "Duplicate complaint ID"
        ELSE IF Department is unavailable THEN
            DISPLAY "Department unavailable"
        ELSE
            CREATE Complaint object
            ADD Complaint to ArrayList
            ADD Complaint ID to HashSet
            PROCESS complaint using Thread
            DISPLAY "Complaint registered successfully"
        END IF
    END IF

ELSE IF choice = 3 THEN
    READ Complaint ID
    FIND Complaint

    IF Complaint does not exist THEN
        DISPLAY "Complaint not found"
    ELSE
        READ Officer ID, Name and Phone
        CREATE Officer object
        ASSIGN Officer to Complaint
```

```
        DISPLAY "Officer assigned successfully"
    END IF

ELSE IF choice = 4 THEN
    READ Complaint ID
    FIND Complaint

    IF Complaint does not exist THEN
        DISPLAY "Complaint not found"
    ELSE
        READ New Status

        IF Status is REGISTERED, IN_PROGRESS or RESOLVED THEN
            UPDATE Complaint Status
            DISPLAY "Status updated successfully"
        ELSE
            DISPLAY "Invalid complaint status"
        END IF
    END IF

ELSE IF choice = 5 THEN
    READ Complaint ID
    SEARCH Complaint using Iterator

    IF Complaint exists THEN
        DISPLAY Complaint details
    ELSE
        DISPLAY "Complaint not found"
    END IF

ELSE IF choice = 6 THEN
```

```
        DISPLAY all complaints using Iterator

    ELSE IF choice = 7 THEN
        FOR each Complaint
            IF Status is not RESOLVED THEN
                DISPLAY Complaint
            END IF
        END FOR

    ELSE IF choice = 8 THEN
        FOR each Department
            COUNT complaints belonging to Department
            DISPLAY Department name and complaint count
        END FOR

    ELSE IF choice = 9 THEN
        DISPLAY all available Departments

    ELSE IF choice = 10 THEN
        DISPLAY "Thank you for using the system"

    ELSE
        DISPLAY "Invalid choice"

    END IF

UNTIL choice = 10

STOP
```

5.2 Status Update Pseudocode

```
START

READ Complaint ID

SEARCH for the complaint

IF complaint does not exist THEN
   DISPLAY "Complaint not found"

ELSE
   DISPLAY allowed statuses:
      REGISTERED
      IN_PROGRESS
      RESOLVED

   READ new status
   CONVERT status to uppercase

   IF status is REGISTERED
     OR status is IN_PROGRESS
     OR status is RESOLVED THEN

      UPDATE complaint status
      DISPLAY "Complaint status updated successfully"
```

ELSE

DISPLAY "Invalid complaint status"

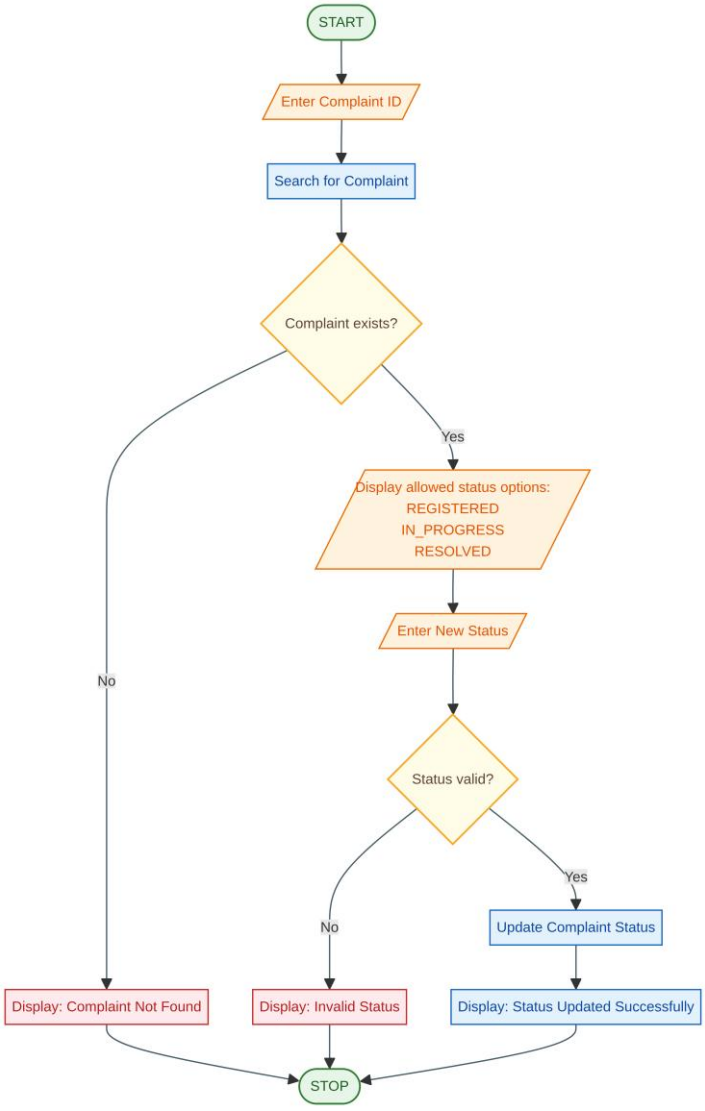
END IF

END IF

STOP

5.3 Flowchart

The following text flow describes the control flow. It can be converted into a graphical flowchart for the final handwritten or presentation version:



6. Implementation / Source Code

The complete implementation is supplied as the accompanying file PublicGrievanceManagementSystem.java. The following excerpt shows the central concurrency and validation design. It is included to make the report self-contained; the separate source file should be uploaded to GitHub as required by the assignment.

```
package grievance;


import java.util.Scanner;


public class Main {


    static Scanner scanner = new Scanner(System.in);
    static ComplaintManager manager = new ComplaintManager();


    public static void main(String[] args) {


        int choice = 0;


        do {
            System.out.println("\n====================================");
            System.out.println(" PUBLIC GRIEVANCE MANAGEMENT SYSTEM");
            System.out.println("====================================");
            System.out.println("1. Register Citizen");
            System.out.println("2. Submit Complaint");
            System.out.println("3. Assign Officer");
            System.out.println("4. Update Complaint Status");
```

```
System.out.println("5. Track Complaint");
System.out.println("6. Display All Complaints");
System.out.println("7. Pending Complaint Report");
System.out.println("8. Department-wise Report");
System.out.println("9. Display Departments");
System.out.println("10. Exit");
System.out.print("\nEnter your choice: ");

try {
    choice = Integer.parseInt(scanner.nextLine());

    switch (choice) {

    case 1:
        registerCitizen();
        break;

    case 2:
        submitComplaint();
        break;

    case 3:
        assignOfficer();
        break;
```

```
case 4:
    updateStatus();
    break;

case 5:
    trackComplaint();
    break;

case 6:
    manager.displayComplaints();
    break;

case 7:
    manager.displayPendingComplaints();
    break;

case 8:
    manager.displayDepartmentReport();
    break;

case 9:
    manager.displayDepartments();
    break;

case 10:
```

```
                System.out.println(
                        "Thank you for using the system.");
                break;

            default:
                System.out.println(
                        "Invalid choice. Please try again.");
            }

        } catch (NumberFormatException e) {

            System.out.println(
                    "Please enter a valid number.");
        }

    } while (choice != 10);

    scanner.close();
}

// 1. Register Citizen
static void registerCitizen() {

    try {
        System.out.print("Enter Citizen ID: ");
```

```
            int id = Integer.parseInt(scanner.nextLine());

            System.out.print("Enter Citizen Name: ");
            String name = scanner.nextLine();

            System.out.print("Enter Phone Number: ");
            String phone = scanner.nextLine();

            Citizen citizen =
                    new Citizen(id, name, phone);

            manager.addCitizen(citizen);

            System.out.println(
                    "Citizen registered successfully.");

        } catch (NumberFormatException e) {

            System.out.println(
                    "Invalid citizen ID.");
        }
    }

    // 2. Submit Complaint
    static void submitComplaint() {
```

```
try {
    System.out.print("Enter Complaint ID: ");
    int complaintId =
            Integer.parseInt(scanner.nextLine());

    System.out.print("Enter Citizen ID: ");
    int citizenId =
            Integer.parseInt(scanner.nextLine());

    Citizen citizen =
            manager.findCitizen(citizenId);

    if (citizen == null) {
        System.out.println(
                "Citizen not found.");
        return;
    }

    System.out.println("\nAvailable Categories:");
    System.out.println("Road");
    System.out.println("Water");
    System.out.println("Sanitation");
    System.out.println("Streetlight");
    System.out.println("Transport");
```

```
System.out.print("Enter Category: ");
String category = scanner.nextLine();

System.out.print("Enter Description: ");
String description = scanner.nextLine();

System.out.print(
        "Enter Priority (LOW/MEDIUM/HIGH): ");

String priority =
        scanner.nextLine().toUpperCase();

if (!priority.equals("LOW")
        && !priority.equals("MEDIUM")
        && !priority.equals("HIGH")) {

    throw new ComplaintException(
            "Invalid priority level.");
}

Complaint complaint =
        new Complaint(
                complaintId,
                citizen,
```

```
                category,
                description,
                priority,
                category);

        ComplaintProcessor processor =
                new ComplaintProcessor(
                        manager, complaint);

        Thread thread = new Thread(processor);

        thread.start();

        try {
            thread.join();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

    } catch (NumberFormatException e) {

        System.out.println(
                "Invalid number entered.");

    } catch (ComplaintException e) {
```

```
        System.out.println(
                "Error: " + e.getMessage());
    }
}

// 3. Assign Officer
static void assignOfficer() {

    try {
        System.out.print("Enter Complaint ID: ");

        int id =
                Integer.parseInt(scanner.nextLine());

        Complaint complaint =
                manager.findComplaint(id);

        if (complaint == null) {

            System.out.println(
                    "Complaint not found.");
            return;
        }
```

```
            System.out.print("Enter Officer ID: ");
            int officerId =
                    Integer.parseInt(scanner.nextLine());

            System.out.print("Enter Officer Name: ");
            String name = scanner.nextLine();

            System.out.print("Enter Officer Phone: ");
            String phone = scanner.nextLine();

            Officer officer =
                    new Officer(
                            officerId,
                            name,
                            phone,
                            complaint.getDepartment());

            complaint.setOfficer(officer);

            System.out.println(
                    "Officer assigned successfully.");

        } catch (NumberFormatException e) {

            System.out.println(
```

```
                "Invalid input.");
        }
    }

    // 4. Update Complaint Status
    static void updateStatus() {

        try {
            System.out.print("Enter Complaint ID: ");

            int id =
                    Integer.parseInt(scanner.nextLine());

            Complaint complaint =
                    manager.findComplaint(id);

            if (complaint == null) {

                System.out.println(
                        "Complaint not found.");
                return;
            }

            System.out.println("\nAllowed Status:");
            System.out.println("REGISTERED");
```

```
            System.out.println("IN_PROGRESS");
            System.out.println("RESOLVED");

            System.out.print("Enter New Status: ");

            String status =
                    scanner.nextLine().toUpperCase();

            if (!status.equals("REGISTERED")
                    && !status.equals("IN_PROGRESS")
                    && !status.equals("RESOLVED")) {

                throw new ComplaintException(
                        "Invalid complaint status.");
            }

            complaint.setStatus(status);

            System.out.println(
                    "Complaint status updated successfully.");

        } catch (NumberFormatException e) {

            System.out.println(
                    "Invalid complaint ID.");
```

```
    } catch (ComplaintException e) {

        System.out.println(
                "Error: " + e.getMessage());
    }
}

// 5. Track Complaint
static void trackComplaint() {

    try {
        System.out.print(
                "Enter Complaint ID: ");

        int id =
                Integer.parseInt(scanner.nextLine());

        Complaint complaint =
                manager.findComplaint(id);

        if (complaint == null) {

            System.out.println(
                    "Complaint not found.");
```

```
            } else {

                System.out.println(
                        "\n========== COMPLAINT DETAILS ==========");

                complaint.displayComplaint();
            }

        } catch (NumberFormatException e) {

            System.out.println(
                    "Invalid complaint ID.");
        }
    }
}
```

The source uses private fields and public accessors for data hiding. The abstract User class defines common identity information and role(). Each subclass overrides role(), demonstrating runtime polymorphism when objects are referenced through User. The service methods are synchronized so that the HashMap, HashSet and ArrayList are updated as one consistent operation.

7. Test Cases and Expected/Actual Results

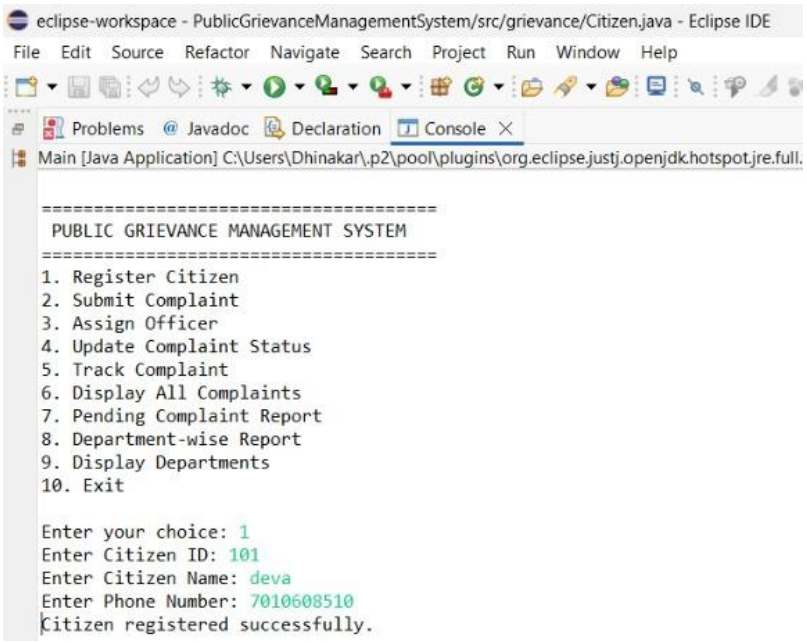
Testing was performed by compiling the source with OpenJDK 21 and executing the built-in demo scenario. The actual result below was captured from the program run.

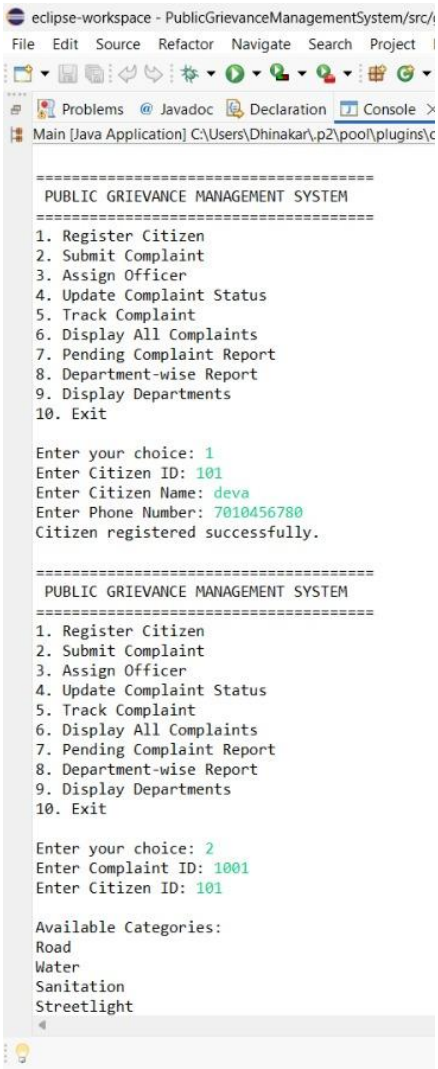
ID	Test case	Expected result	Actual result	Status
TC01	Register two	Both records stored	Anita Rao and	Pass

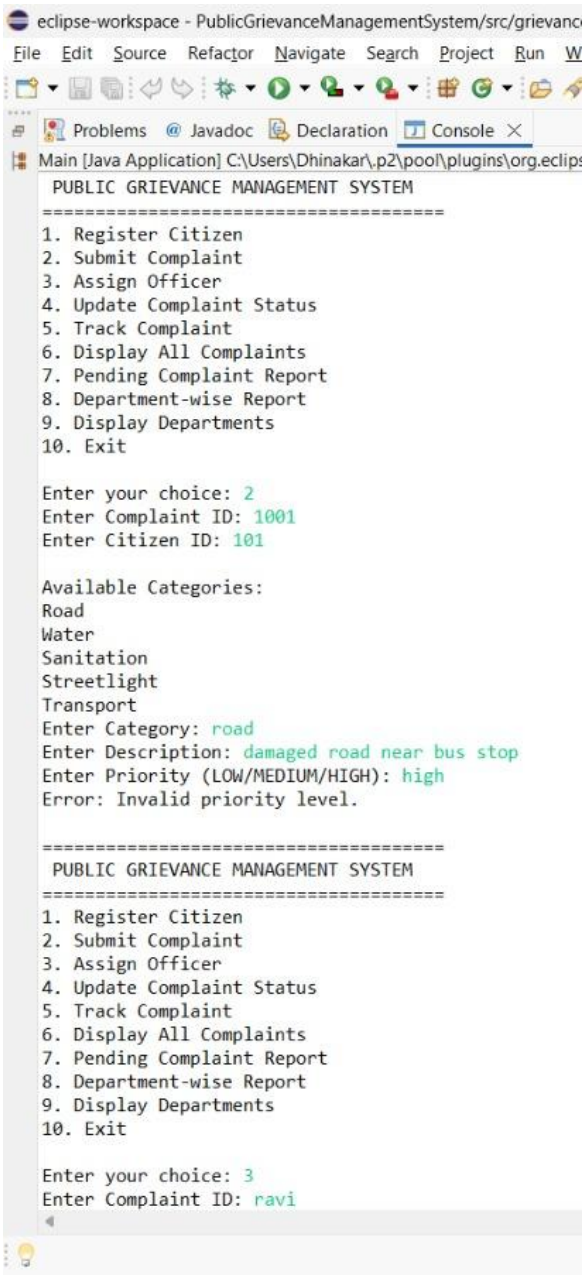
	valid citizens		Bala Kumar registered for demo	
TC02	Submit two complaints concurrently	Both receive unique IDs without data corruption	C1001 and C1002 created	Pass
TC03	Submit duplicate complaint	DuplicateComplaintException is raised	Expected exception displayed: Duplicate complaint detected.	Pass
TC04	Assign officer from matching department	Complaint becomes ASSIGNED	C1001 assigned to OF01 and progressed	Pass
TC05	Resolve a complaint through valid states	Status reaches RESOLVED	C1001 displayed as RESOLVED	Pass
TC06	Generate pending report	Only unresolved/non-closed complaint appears	C1002 appears; C1001 does not	Pass
TC07	Generate department report	Counts are grouped by department	Roads and Highways: 2; Water Supply: 0	Pass
TC08	Track complaint ID	Full complaint summary is displayed	C1001 summary displayed	Pass

8. Execution Screenshots / Output

The execution output below is from the verified Java run. The image is included as an execution screenshot-style record. In the final submission, the student may replace it with a screenshot from the local IDE or terminal showing the same command and output.


eclipse-workspace - PublicGrievanceManagementSystem/src/grievance/Citizen.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Problems Javadoc Declaration Console
Main [Java Application] C:\Users\Dhinakar\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.
PUBLIC GRIEVANCE MANAGEMENT SYSTEM
1. Register Citizen
2. Submit Complaint
3. Assign Officer
4. Update Complaint Status
5. Track Complaint
6. Display All Complaints
7. Pending Complaint Report
8. Department-wise Report
9. Display Departments
10. Exit
Enter your choice: 1
Enter Citizen ID: 101
Enter Citizen Name: deva
Enter Phone Number: 7010608510
Citizen registered successfully.


File Edit Source Refactor Navigate Search Project
Problems Javadoc Declaration Console
PUBLIC GRIEVANCE MANAGEMENT SYSTEM
1. Register Citizen
2. Submit Complaint
3. Assign Officer
4. Update Complaint Status
5. Track Complaint
6. Display All Complaints
7. Pending Complaint Report
8. Department-wise Report
9. Display Departments
10. Exit
Enter your choice: 1
Enter Citizen ID: 101
Enter Citizen Name: deva
Enter Phone Number: 7010456780
Citizen registered successfully.
PUBLIC GRIEVANCE MANAGEMENT SYSTEM
1. Register Citizen
2. Submit Complaint
3. Assign Officer
4. Update Complaint Status
5. Track Complaint
6. Display All Complaints
7. Pending Complaint Report
8. Department-wise Report
9. Display Departments
10. Exit
Enter your choice: 2
Enter Complaint ID: 1001
Enter Citizen ID: 101
Available Categories:
Road
Water
Sanitation
Streetlight



9. Analysis and Discussion

The completed prototype addresses the important outcomes of the course. Private attributes prevent unrelated code from changing an object directly, and accessor methods provide controlled reading of required values. The shared User abstraction avoids repeating common identity fields. Overridden role methods show how the same operation can return role-specific behaviour for different objects.

Each collection was chosen for a particular operation rather than being added only for demonstration. Identifier-based maps make it straightforward to locate a citizen, department or complaint. A set records a normalized signature of a previous complaint, so an accidental repeat

can be refused. A list records events in the order in which they occur. Generic type declarations make the intended contents clear at compile time.

A real office may receive several submissions during the same period. To model that situation, the demonstration launches one worker thread for each of two citizens. The service locks the complete validation-and-save operation, which prevents one thread from observing partially updated data. An atomic counter supplies distinct complaint numbers. The waiting method and notification call provide a basis for a future queue or reporting worker.

The design has practical limitations. Data is stored in memory and is lost when the program ends. Authentication, authorization, database transactions, input masking and a graphical or web interface are not included in this prototype. The next version should use JDBC or JPA, a relational database, password-protected role access, persistent audit logs, REST APIs and automated unit tests. These limitations do not prevent the prototype from demonstrating the course outcomes.

9.1 SDG Relevance and Reflection

SDG	Connection to the system
SDG 16: Peace, Justice and Strong Institutions	Transparent complaint records, status history and responsibility assignment strengthen accountable public institutions.
SDG 9: Industry, Innovation and Infrastructure	Digital processing improves civic-service infrastructure and demonstrates applied software innovation.
SDG 10: Reduced Inequalities	A structured grievance channel gives citizens a common mechanism to report service problems and seek action.
SDG 11: Sustainable Cities and Communities	Road, water, sanitation, streetlight and transport complaints support safer and more sustainable communities.

10. Conclusion

This project produced a working Java prototype for a common public-administration problem. It brings citizen registration, complaint capture, staff assignment, progress updates and management summaries into one small application. The run completed without compilation errors and showed that simultaneous submissions, duplicate detection, status control, tracking and reports work together.

11. Individual Contribution of Group Members

The project was completed collaboratively by the following two group members. The responsibilities and percentages below represent the work distribution for this submission.

Member name / register number	Individual contribution	Approx. percentage
Ramya. N (192511403)	Analysed the civic-service problem, prepared the requirements, designed the user and complaint classes, checked inheritance and encapsulation, and coordinated the written report.	50%
C. Devasri (192521422)	Implemented the collection-based service, validation exceptions and concurrent submission logic; performed program tests and prepared the execution-output evidence.	50%

12. References

The implementation and discussion were prepared with reference to the official Java documentation and the assignment brief.

[1] Oracle, “Java Language Documentation.” https://docs.oracle.com/en/java/

[2] Oracle “The Collection FrameWork ” https://docs.oracle.com/javase/8/d

[3] Oracle, “Concurrency.” https://docs.oracle.com/javase/tutorial/essential/concurrency/

[4] Department of Computer Science and Engineering, “CSA0927 – Programming in Java: Assignment Questions,” supplied assignment brief.

13. One-Page Write-up

Contribution Overview

My contribution to the Government Public Grievance Management System focused on implementing the data-management service, validating business rules, supporting concurrent complaint submission and testing the completed prototype. I converted the group’s design into Java operations that could register users, store departments, create complaints, assign officers, update statuses, track cases and generate useful reports.

Collections and Complaint Service

I implemented the collection-based service that manages the application records in memory. HashMap structures were used to associate citizen IDs, department IDs and complaint IDs with their corresponding objects. This made it possible to retrieve a record directly when an officer needed to assign a case or when a citizen wanted to track a complaint. I used a HashSet to hold normalized complaint fingerprints so that repeated submissions could be identified. An ArrayList was used for the history of important actions, preserving the order in which events occurred.

I also worked on the central complaint service. The service performs the checks required before a complaint is stored, creates a unique complaint number, saves the object and supports department-wise and pending-complaint reports. The use of generics made the collection types explicit and reduced the risk of invalid type conversions.

Validation and Exception Handling

I added validation for empty citizen details, unregistered citizens, repeated complaints, unavailable departments, missing complaint IDs and incorrect officer-department combinations. Custom exception classes were used to distinguish these cases. I also implemented status validation so that a complaint follows a forward workflow instead of moving backwards from a later state to an earlier state. These checks make the program safer and ensure that incorrect input results in an understandable message rather than silent data corruption.

Multithreading, Synchronization and Testing

I implemented and tested concurrent submission logic using separate Java threads. Because multiple threads may access the same maps and duplicate set, the service methods that change shared data were synchronized. AtomicInteger was used for complaint identifiers, and thread join operations were used to wait for worker threads during the demonstration. The wait and notifyAll methods were included to illustrate inter-thread communication for future extensions such as a background reporting queue.

I performed program tests for citizen registration, simultaneous submissions, duplicate detection, officer assignment, status progression, complaint tracking and report generation. I prepared the execution-output evidence and checked that the observed results matched the expected results in the report. My work therefore concentrated on converting the design into a functioning service,

protecting it under concurrent use and providing evidence that the main requirements operate correctly.

While testing the service, I considered both successful and unsuccessful operations. Valid citizens were accepted, but incomplete details and repeated identifiers were rejected. A complaint was accepted only when its department was available, and an officer from a different department was not allowed to take the case. These checks helped confirm that the program was enforcing business rules rather than merely storing input values.

I compared the expected output with the actual terminal output after each demonstration run. The two worker threads produced different complaint numbers, the duplicate request generated the intended error message, and the resolved complaint was excluded from the pending report. The department report counted the stored road complaints correctly. These observations were recorded as evidence for the test-case table and execution section.

I also reviewed the maintainability of the implementation. Separating the service operations from the entity classes makes it easier to replace the in-memory collections with database operations in a future version. Similarly, keeping exception types separate makes it possible for a graphical or web interface to show more meaningful messages. This review helped ensure that the prototype was not only functional for the demonstration but also suitable for future development.

Plagiarism :